

PCS956-TS-companion_B

September 18, 2026

0.1 PCS956 time series companion B: Classical models, forecasting baselines, ML on lagged features, temporal validation, spectral views

This companion notebook provides **code templates** for the topics discussed in lecture TS2:

- forecasting baselines: mean, persistence, seasonal naive, simple AR;
- classical models: AR/MA/ARIMA as baselines with residual checks;
- time series as supervised learning with lagged features;
- simple ML models (Random Forest) on lagged features;
- time-aware train/validation/test splits (with optional gaps);
- evaluation on levels vs differences;
- periodograms for spectral views of seasonality.

The notebook is intended as a **starting point** for the mini-project. You are expected to:

- adapt the templates to your own dataset and question;
- make sensible choices of train/validation/test splits;
- compare ML models against simple baselines under temporal validation;
- interpret metrics (especially R^2) critically in light of autocorrelation and random-walk behaviour.

0.1.1 Available datasets and how Companion B relates to Companion A

The example time series used in this module are introduced in **Companion A**. They are exported from the `astsa` R package and stored in a local `data/` folder:

- `../data/soi.csv` – Southern Oscillation Index (monthly, univariate)
- `../data/ENSO.csv` – ENSO index (monthly, multivariate)
- `../data/gtemp_both.csv` – global temperature anomalies (annual, univariate)
- `../data/djia.csv` – Dow Jones Industrial Average (daily, multivariate)
- `../data/eqexp.csv` – earthquake vs explosion traces (classification-style)

In **Companion A** you:

- inspect these example datasets (time index, sampling pattern, data quality),
- explore levels, differences, rolling summaries, and ACF/PACF,
- perform simple decompositions and persistence baselines.

In **Companion B** we assume that basic inspection has already been done and focus on:

- forecasting baselines on these series (mean, persistence, seasonal naive, simple AR),
- ARIMA-type models as classical baselines,
- supervised-learning formulations with lagged features,

- time-aware train/validation/test splits and evaluation on levels vs differences.

For the mini-project you are encouraged, where possible, to work with a **sharable time-series dataset** that is relevant to your interests or domain. This may be your own research data (if suitable for sharing), an openly available dataset, or a synthetic/anonymised variant of restricted data.

- run the templates end-to-end on known series, and
- then replace `read_this_file.csv` with `read_my_file.csv` in your own notebook.

If you do use one of the **astsa**-derived datasets (for example SOI, ENSO, global temperature, or DJIA) in your mini-project:

- remember that the period covered is **historical** and not up to date; this is fine for the mini-project, but you should mention it clearly in your report;
- reuse the inspection patterns from Companion A before fitting models in Companion B;
- document your choices of train/validation/test splits, seasonal period, and lag structure.

You are also welcome to explore **other datasets from the astsa package** beyond the examples provided here (for instance, additional climate, economic, or signal series). If you do so, treat them in the same way:

- make it clear that they are curated, historical teaching datasets rather than current data;
- describe the time coverage and any limitations this imposes on the questions you can address.

0.1.2 1. Imports and basic setup

We start by importing standard libraries. The plotting style is set for readability.

0.1.3 2. Example series and temporal splits (with optional gap)

In the mini-project you will work with a **time-series dataset** that is suitable for sharing in this course. This can be a real series from an openly available source (for example public climate, energy, health, or finance data), a series from your own work that you are allowed to share, or a synthetic/anonymised series that you have constructed to mimic restricted data. You should only use data that you are allowed to share: do not use confidential or sensitive data (for example identifiable patient records) in this notebook.

If you want to explore methods on restricted data, a good approach is to construct a **synthetic or anonymised variant** that preserves the main statistical features but does not expose individual-level information. The `data/` folder in this repository contains example series that you can use to test and understand the code templates before plugging in your own data or synthetic versions of it.

Here we define a simple synthetic example series with:

- a weak trend,
- weekly seasonality (daily data),
- some noise.

We also define a utility function for time-aware splitting into train, validation, and test segments, with an optional **gap** as discussed in TS1 (Section 4.4).



0.1.4 3. Forecasting baselines: mean, persistence, seasonal naive, simple AR

Baselines are central tools, not toy examples. They answer:

How much can we achieve with very simple structure, before we try complex ML models?

We implement:

- **mean baseline** – predict the training mean;
- **persistence** – predict tomorrow as today;
- **seasonal naive** – repeat the last season;
- **simple AR(p)** – linear autoregression with a few lags.

Persistence (test): MAE=1.105, RMSE=1.309, R2=0.109

Seasonal naive (test): MAE=0.599, RMSE=0.734, R2=0.722

AR(2) baseline (test): MAE=2.023, RMSE=2.414, R2=-2.012

0.1.5 4. ARIMA baseline and residual diagnostics

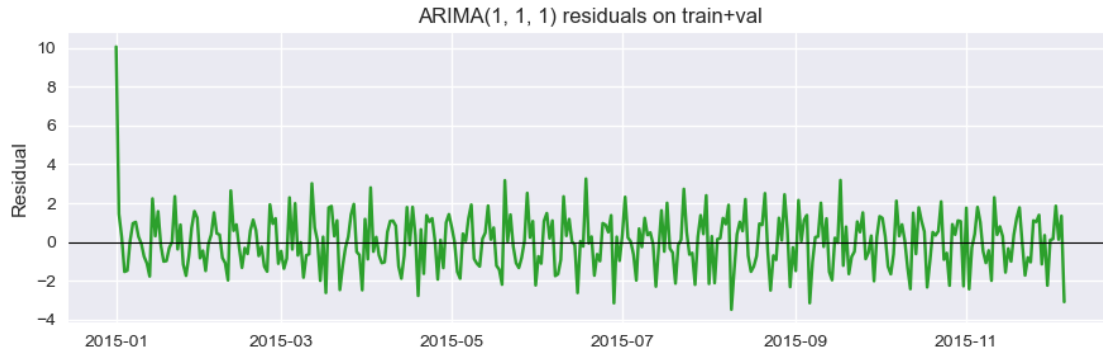
Classical ARIMA models impose linear structure on lagged values and differences.

Here we use ARIMA as a **baseline**, not as a magical final model.

We:

- fit an ARIMA(p, d, q) model to train+validation;
- forecast over the test window;
- inspect residuals.

ARIMA(1, 1, 1) (test): MAE=1.663, RMSE=2.110, R2=-1.301



TODO (later): Add ACF plots for residuals and a small AIC-based grid search for ARIMA orders.

0.1.6 5. Time series as supervised learning with lagged features

Many forecasting problems can be viewed as supervised learning:

- inputs: lagged values and possibly exogenous variables;
- targets: future values at a chosen horizon.

We implement:

- a function to convert a univariate series into (X, y) pairs using lag windows;
- a helper that builds train/validation/test splits in supervised form;
- a simple RandomForestRegressor as an ML model on lagged features.

Random Forest on lagged features:

RF (validation): MAE=0.540, RMSE=0.666, R2=0.775

RF (test): MAE=0.584, RMSE=0.744, R2=0.720

0.1.7 6. Levels vs differences: templates for evaluation

For highly autocorrelated series (including random-walk-like behaviour), models can look very strong on **levels** simply by exploiting persistence.

To see whether we are capturing genuine structure beyond persistence, it is useful to:

- compare performance on **levels** vs **differences**;
- compare models to baselines (mean, persistence, seasonal naive, simple AR, ARIMA).

We define a simple differencing function and reuse the supervised learning pipeline on the differenced series.

Levels (original series):

RF on levels (test): MAE=0.584, RMSE=0.744, R2=0.720

Persistence on levels (test): MAE=1.105, RMSE=1.309, R2=0.109

Differences (first differences of series):

RF on differences (test): MAE=0.582, RMSE=0.730, R2=0.701

Persistence on differences (test): MAE=1.182, RMSE=1.503, R2=-0.301

Interpretation for the mini-project:

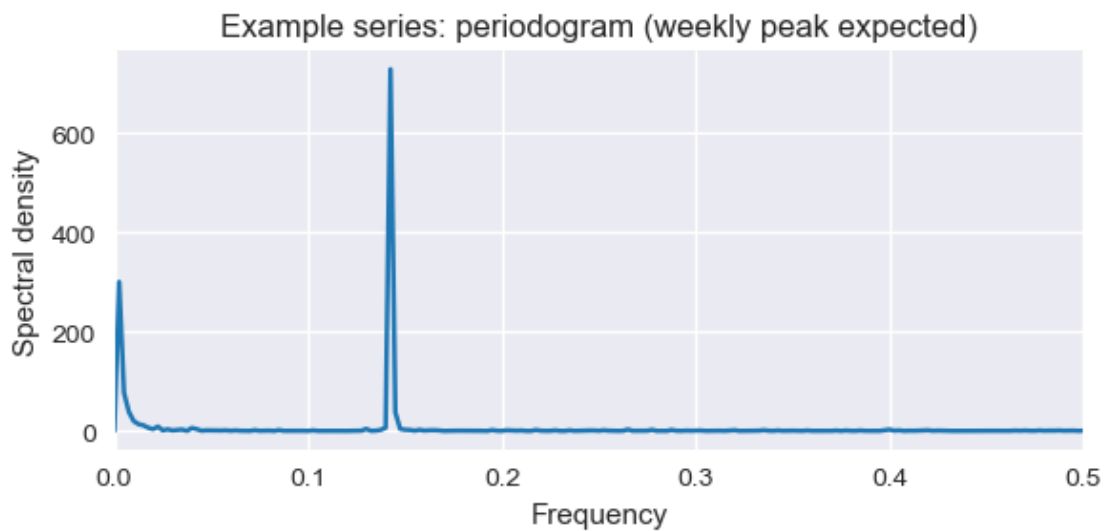
- If a model looks strong on levels but shows no improvement over baselines on differences, it may simply be reproducing persistence and trend.
- Comparing levels vs differences helps reveal whether there is exploitable structure beyond simple random-walk-like behaviour.

0.1.8 7. Spectral view: periodograms for seasonality and structure

Spectral density and periodograms show how variance is distributed across frequencies:

- peaks indicate strong periodic components (e.g. weekly cycles in daily data);
- relatively flat spectra are closer to white noise;
- spectral views complement autocorrelation plots and ARIMA models.

We implement a simple periodogram helper and apply it to the example series.



In your mini-project you can:

- use periodograms (and ACFs from TS1) to confirm suspected seasonality;
- choose seasonal periods for seasonal naive baselines and seasonal components in ARIMA;
- contrast series with strong peaks (clear seasonality) against series that behave more like random walks.

0.1.9 8. Mini-project scaffolding: comparing baselines and models

Finally, we provide a small helper that:

- runs persistence, seasonal naive (optional), ARIMA, and Random Forest on a series;
- evaluates all models on the same test window;
- returns a dictionary of metrics.

This is intended as a **copy-paste template** for your own data. You should:

- adjust `train_end`, `val_end`, `n_lags`, `seasonal_period`, and ARIMA order to suit your series;
- document and justify these choices in your mini-project notebook.

Persistence (test): MAE=1.105, RMSE=1.309, R2=0.109

Seasonal naive (m=7): MAE=0.599, RMSE=0.734, R2=0.722

ARIMA(1, 1, 1) (test): MAE=1.663, RMSE=2.110, R2=-1.301

RF on lagged features (test): MAE=0.584, RMSE=0.744, R2=0.720

	model_key	label	MAE	RMSE	R2
0	persistence	Persistence (test)	1.105059	1.308940	0.108834
1	seasonal	Seasonal naive (m=7)	0.598921	0.733574	0.721857
2	arima	ARIMA(1, 1, 1) (test)	1.663370	2.109948	-1.301036
3	rf	RF on lagged features (test)	0.584476	0.743917	0.719688

0.1.10 9. How to adapt this notebook for your mini-project

For your own dataset:

- Replace `series_example` with a series loaded from a dataset that is suitable for this course (for example your own research data that you can share, an openly available dataset, or a synthetic/anonymised variant of restricted data). The example files in `data/` are there so you can test this pipeline before applying it to your chosen time series.
- Choose sensible `train_end`, `val_end`, and `gap` based on the length of your series and concern about long-range dependence.
- Use EDA tools from Companion A (plots, differences, simple decomposition) to understand trend, seasonality, and anomalies before modelling.
- Use:
 - persistence and seasonal naive baselines,
 - at least one classical model (e.g. ARIMA),
 - at least one ML model on lagged features (e.g. RF), and compare them under temporal validation.
- Evaluate both on **levels** and, where meaningful, on **differences** or residuals.
- Interpret R^2 and other metrics critically:
 - negative test R^2 means worse than a simple mean baseline on that target;
 - strong performance on levels only may reflect persistence rather than genuine skill.

A clear description of what you tried, what you found, and what the limitations are (including cases where no improvement over simple baselines is seen) is more important than impressive headline scores.

0.1.11 10. Using a real dataset: SOI as a forecasting example

In the mini-project you are encouraged to work with a **sharable time-series dataset** that is relevant to your interests or domain. The SOI, ENSO, global temperature, DJIA, and eqexp series in the `data/` folder are provided as **example data** for testing and illustrating the workflow.

This section shows how to:

- load the SOI series from CSV (paths and columns as in Companion A),
- set a time index and check basic structure,

- plug SOI into the same baseline/ARIMA/ML comparison pipeline used for the synthetic example.

Remember:

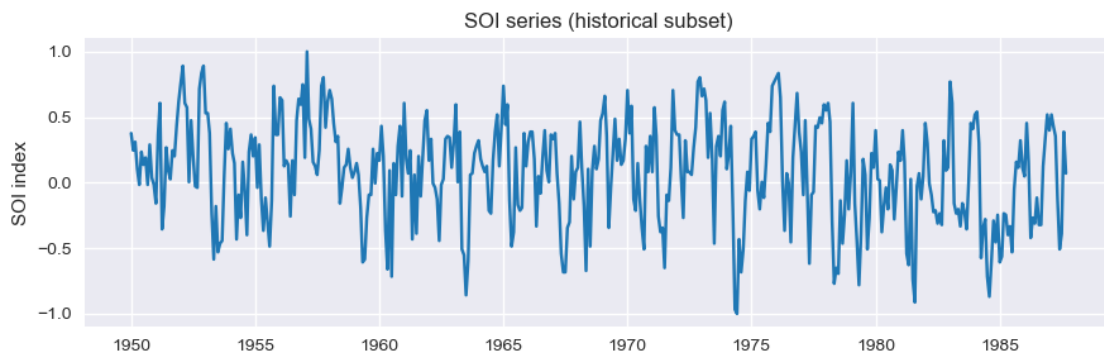
- the SOI data provided here are **historical** (they are exported from the `astsa` R package) and are not up to date;
- this is perfectly fine for the mini-project, but you should mention this clearly in your report if you use SOI as one of your series.

SOI head:

```
date
1950-01-01    0.377
1950-02-01    0.246
1950-03-01    0.311
1950-04-01    0.104
1950-05-01   -0.016
Name: SOI, dtype: float64
```

SOI info:

```
<class 'pandas.DataFrame'>
DatetimeIndex: 453 entries, 1950-01-01 to 1987-09-01
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    SOI      453 non-null      float64
dtypes: float64(1)
memory usage: 7.1 KB
None
```



10.1 Temporal split and baseline comparison for SOI We now:

- choose a simple train/validation/test split on SOI,
- treat the data as monthly with a seasonal period of 12,
- compare persistence, seasonal naïve, ARIMA, and Random Forest on lagged features.

These choices are deliberately simple; for the mini-project you should adapt the split points, seasonal period, and lag length to your own question and dataset.

```
Persistence (test): MAE=0.230, RMSE=0.304, R2=0.322
Seasonal naive (m=12): MAE=0.345, RMSE=0.431, R2=-0.367
ARIMA(1, 1, 1) (test): MAE=0.348, RMSE=0.402, R2=-0.189
```

```
C:\Users\laj\.conda\envs\pcs956\Lib\site-
packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency
information was provided, so inferred frequency MS will be used.
    self._init_dates(dates, freq)
C:\Users\laj\.conda\envs\pcs956\Lib\site-
packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency
information was provided, so inferred frequency MS will be used.
    self._init_dates(dates, freq)
C:\Users\laj\.conda\envs\pcs956\Lib\site-
packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency
information was provided, so inferred frequency MS will be used.
    self._init_dates(dates, freq)
```

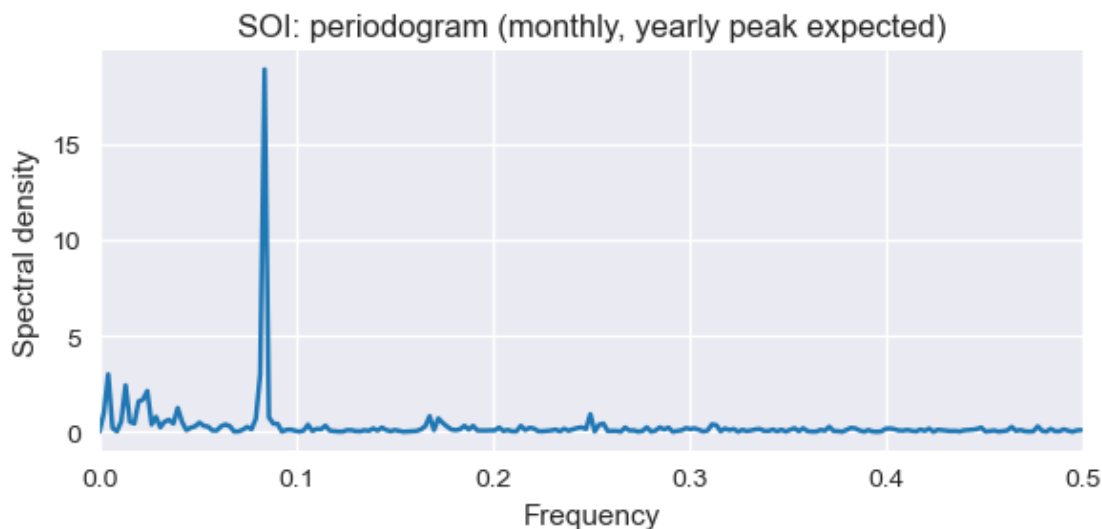
```
RF on lagged features (test): MAE=0.237, RMSE=0.299, R2=0.373
```

	model_key	label	MAE	RMSE	R2
0	persistence	Persistence (test)	0.229761	0.303851	0.322241
1	seasonal	Seasonal naive (m=12)	0.344618	0.431466	-0.366964
2	arima	ARIMA(1, 1, 1) (test)	0.347880	0.402388	-0.188923
3	rf	RF on lagged features (test)	0.236890	0.299460	0.372651

10.2 SOI: levels vs differences and spectral view For highly autocorrelated series such as SOI, it is important to:

- compare performance on **levels** versus **first differences**,
- look at the spectrum to understand dominant periodicities (for example, annual structure).

Below we show the periodogram of SOI and a simple comparison of persistence vs RF on first differences.



SOI first differences:

RF on SOI differences (test): MAE=0.232, RMSE=0.308, R2=-0.022

Persistence on SOI differences (test): MAE=0.357, RMSE=0.436, R2=-1.055

Interpretation:

- if a model performs well on SOI **levels** but not on **differences**, it may simply be exploiting persistence and seasonal structure;
- genuine improvements over baselines on **differences** are stronger evidence that the model is capturing additional structure rather than just copying the recent past.

0.1.12 11. Spectral comparison: earthquake vs explosion traces (eqexp)

The **eqexp** dataset in Companion A contains **seismic traces** from earthquakes and explosions. Here we do not build a full classifier; instead we:

- select one earthquake channel (for example **EQ1**) and one explosion channel (**EX1**),
- estimate their spectral densities using periodograms (or Welch's averaged periodogram),
- compare the typical frequency content of earthquake and explosion signals.

This illustrates how spectral methods can help distinguish different types of time-series signals.

eqexp info:

```
<class 'pandas.DataFrame'>
```

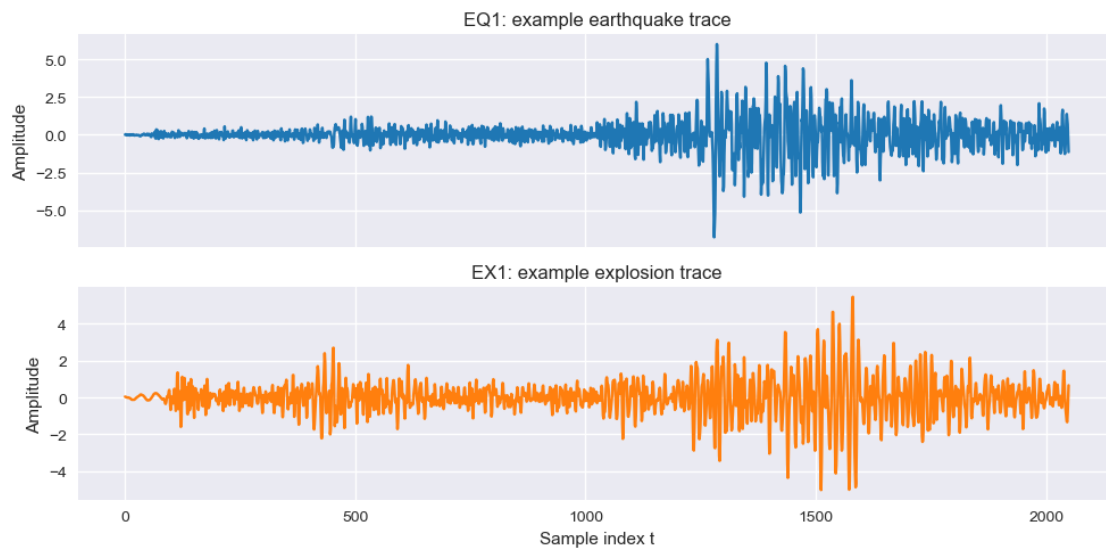
```
RangeIndex: 2048 entries, 1 to 2048
```

```
Data columns (total 17 columns):
```

#	Column	Non-Null Count	Dtype
0	EQ1	2048 non-null	float64
1	EQ2	2048 non-null	float64
2	EQ3	2048 non-null	float64
3	EQ4	2048 non-null	float64

4	EQ5	2048 non-null	float64
5	EQ6	2048 non-null	float64
6	EQ7	2048 non-null	float64
7	EQ8	2048 non-null	float64
8	EX1	2048 non-null	float64
9	EX2	2048 non-null	float64
10	EX3	2048 non-null	float64
11	EX4	2048 non-null	float64
12	EX5	2048 non-null	float64
13	EX6	2048 non-null	float64
14	EX7	2048 non-null	float64
15	EX8	2048 non-null	float64
16	NZ	2048 non-null	float64

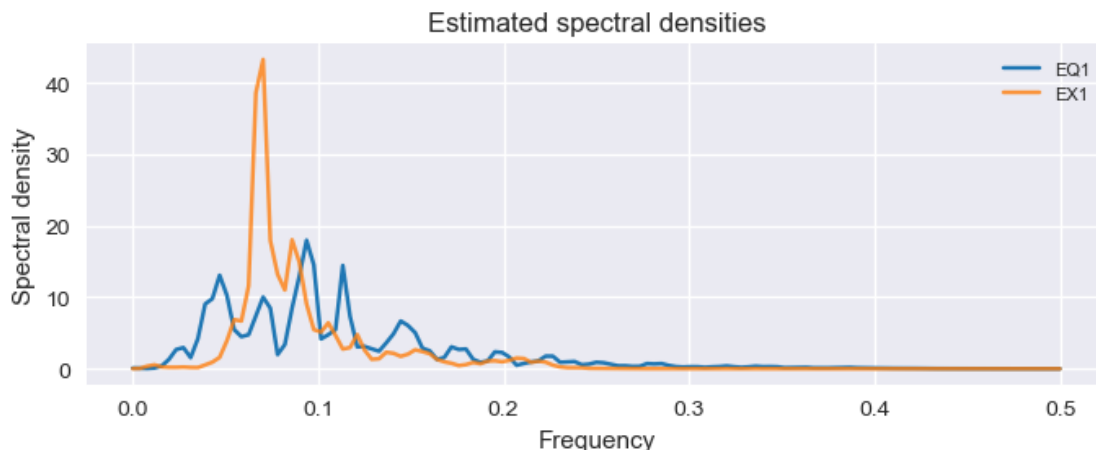
dtypes: float64(17)
memory usage: 272.1 KB
None



11.1 Estimated spectral densities for EQ1 vs EX1 We now estimate spectral densities for EQ1 and EX1. For simplicity we use:

- a standard periodogram via `scipy.signal.periodogram`,
- or, optionally, Welch's method (`scipy.signal.welch`) to obtain a slightly smoother estimate.

The goal is to see whether earthquakes and explosions concentrate their energy in different frequency bands.



Interpretation (for discussion in the mini-project):

- Compare how much energy is concentrated at low vs high frequencies for earthquakes (EQ1) and explosions (EX1).
- Note any dominant peaks or bands where one type of trace clearly differs from the other.
- Think about how simple spectral features (for example, total power in low-frequency vs high-frequency bands) could be used as inputs to a classifier, without going into the full complexity of real seismic-signal processing.

In the mini-project you are **not** expected to build a complete classifier for eqexp. The purpose of this section is to:

- illustrate how spectral methods can distinguish different types of signals,
- give you a concrete starting point for using estimated spectral densities as features or for qualitative comparisons between groups of time-series traces.

0.1.13 12. Future extensions and more advanced templates (for later refinement)

This companion notebook focuses on a **minimal but robust** forecasting toolkit:

- strong baselines (mean, persistence, seasonal naive, simple AR);
- classical ARIMA models used as baselines with residual checks;
- a simple tree-based ML model (Random Forest) on lagged features;
- temporal splits, levels vs differences, and basic spectral views.

For many mini-projects this is sufficient. However, some students may need **more advanced templates**. Possible extensions for later refinement of this notebook include:

- **Multi-step forecasting strategies:**
 - direct multi-step models (separate models for horizon $h = 1, 2, \dots, H$);
 - recursive strategies (iterating 1-step models to produce H -step paths);
 - examples showing how baselines and RF behave for $H > 1$.
- **Exogenous variables and multivariate inputs:**
 - extending `series_to_supervised` / `build_supervised_split` to accept additional exogenous features (for example weather, prices, or other related series);

- simple examples of aligning exogenous inputs in time and avoiding leakage.
- **Residual diagnostics for classical models:**
 - ACF plots for ARIMA residuals;
 - simple tests (for example Ljung–Box) to check remaining autocorrelation;
 - small AIC/BIC-based grids over (p, d, q) to illustrate order selection.
- **Additional ML models on lagged features:**
 - linear/regularised models (for example Ridge or Lasso) as transparent baselines;
 - gradient boosting methods (for example `HistGradientBoostingRegressor` or `XGBoost`) as stronger tree-based benchmarks;
 - simple neural-network templates (for example MLPs on lag windows) for students in more ML-oriented projects.

These extensions are **not required** for the mini-project, but may be added in later versions of the companion notebook or explored independently by interested students. The core methodological stance remains the same:

- start from data quality and EDA (Companion A and TS1);
- use strong baselines and classical models as reference points;
- introduce ML models carefully, with time-aware validation and critical interpretation of metrics;
- document clearly what works, what does not, and why.